

NYC Crime Intelligence

Model, Training, and Dashboard

Mert Samet Kayacıoğlu

Technical Project Report

Aggregate analytics only

July 19, 2026

Executive Summary

NYC Crime Intelligence is an aggregate-only analytical system built from the official NYPD Complaint Data Historic dataset. The project converts a reviewed 10,071,507-row source snapshot into deterministic weekly aggregates, evaluates transparent forecasting rules, produces retrospective hotspot and anomaly signals, and presents the results in a local React dashboard. The system is designed for exploratory analysis of reported complaint patterns, not for person-level prediction or operational policing decisions [2, 1].

10.07M	8,466	0.4894	6
source rows	aggregate segments	ML backtest MAE	dashboard experiences

Core result

The selected `duckdb_lag_ensemble_regressor` slightly outperforms the best deterministic baseline on the recorded time-based backtest: MAE decreases from 0.4929 to 0.4894 and RMSE from 1.4128 to 1.3943. The improvement is small, the two summaries have different prediction coverage, and the model emits point estimates without prediction intervals. The dashboard therefore presents model outputs together with historical error context and explicit limitations.

The report is organized in two parts. Part I documents the data foundation, model design, training logic, leakage controls, backtest results, and governance constraints. Part II explains the dashboard architecture, analytical views, filter behavior, accessibility design, deployment workflow, and verification strategy.

Contents

Executive Summary	i
I Model Development and Training	1
1 Problem Definition and Data Foundation	2
1.1 Analytical objective	2
1.2 Source snapshot and eligibility rule	2
1.3 Weekly panel construction	3
2 Baseline and Lag-Ensemble Model	4
2.1 Deterministic baselines	4
2.2 Feature engineering	4
2.3 Model identity and prediction rule	5
2.4 Training and evaluation chronology	5
3 Evaluation, Interpretation, and Governance	7
3.1 Backtest results	7
3.2 Leakage controls and reproducibility	7
3.3 Limitations	8
3.4 Privacy and responsible use	8
II Dashboard Design and Implementation	9
4 Dashboard Architecture	10
4.1 Technology stack and deployment model	10
4.2 Browser-safe data contracts	10
4.3 Spatial reference and map resilience	11
5 Analytical Views and Interaction Design	12
5.1 Overview	12
5.2 Map workspace: Hotspots, Forecast, and Expected Change	13
5.2.1 Hotspots	13
5.2.2 Forecast	13
5.2.3 Expected Change	13
5.3 Anomalies	14
5.4 Governance	14
6 Accessibility, Verification, and Operations	15
6.1 Accessible and responsive behavior	15
6.2 Automated verification	15

6.3	Refresh and release workflow	16
6.4	Dashboard limitations	16
Conclusion		17
A Reproduction Commands		18
A.1	Python environment	18
A.2	Full analytical rebuild	18
A.3	Verification and dashboard startup	18

Part I

Model Development and Training

Chapter 1

Problem Definition and Data Foundation

1.1 Analytical objective

The forecasting task is to estimate the next week’s reported complaint count for each aggregate segment:

$$\text{week_start} \times \text{borough} \times \text{precinct} \times \text{offense_type} \times \text{law_category}. \quad (1.1)$$

The target is a count of date-eligible source rows, not a count of unique incidents, people, victims, harms, or causal effects. The forecast unit is an area/category segment. Exact addresses, exact event coordinates, complaint identifiers, names, and victim or suspect demographic fields are outside the modeling scope [4, 2].

The project addresses three aggregate questions:

1. How does reported complaint volume vary over time and across geography and offense categories?
2. Which already-observed aggregates show unusual concentration or a high deviation from prior-only history?
3. For one fixed week, how does an explainable lag-based estimate compare with a transparent historical baseline?

1.2 Source snapshot and eligibility rule

The analytical source is NYC Open Data’s *NYPD Complaint Data Historic* dataset, identifier `qgea-i56i`. The reviewed local CSV is 3.19 GB and has SHA-256 checksum `759016def1c04aafaaeaa8e35c622d13abdd4af82a69f7b9e2b5549c08e47827`. The exact source bytes are excluded from Git; a newer portal export constitutes a different analytical snapshot [3].

The deterministic cleaning stage reads reviewed columns as text, normalizes null-like tokens, trims and standardizes categorical values, parses dates with `%m/%d/%Y`, and uses safe numeric casts. A row enters the aggregate dataset only if its canonical complaint start date:

- parses successfully;
- is on or after January 1, 2006; and
- is on or before the explicit review date, July 4, 2026.

Other data-quality conditions are retained as audit flags. Missing dimensions do not by themselves remove a date-eligible row: missing borough, precinct, or offense values are represented by the literal category `UNKNOWN`.

Table 1.1: Reviewed source and aggregate populations.

Population	Rows
Source rows evaluated	10,071,507
Rows included in aggregates	10,049,687
Rows excluded by date eligibility	21,820
Weekly aggregate rows	1,761,447
Distinct aggregate segments used by forecasting	8,466

Quality flags can overlap: 52,050 rows have at least one listed flag and 605 have more than one. Consequently, individual flag counts must not be summed or equated with the 21,820 date-based exclusions [5].

1.3 Weekly panel construction

DuckDB’s `date_trunc('week', ...)` creates Monday-starting buckets. The eligible event dates run from January 1, 2006 through December 31, 2025, while bucket labels run from December 26, 2005 through December 29, 2025. The first label is earlier than event coverage because January 1, 2006 is a Sunday in the week beginning December 26, 2005.

Missing segment-weeks are filled with zero only after the segment first appears. This rule creates a complete post-appearance panel without inventing an earlier history. It is useful for lag calculations but cannot distinguish a true zero from an upstream reporting absence.

The final bucket, beginning December 29, 2025, contains only Monday through Wednesday. It is excluded from baseline backtesting and anomaly scoring, yet it is retained as the latest observation for the fixed forecast. This asymmetry is intentional, but it can depress lag-based inputs relative to a complete seven-day week.

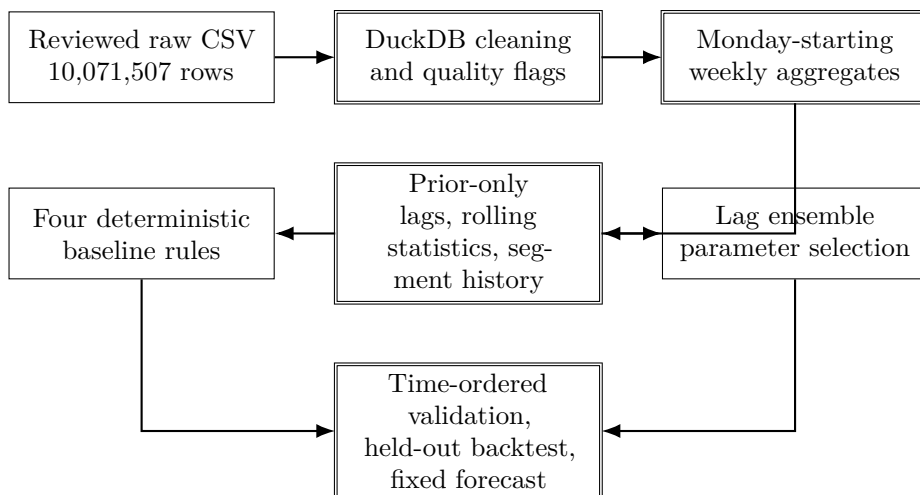


Figure 1.1: Model development pipeline. Every model input is derived from aggregate history and uses prior weeks only.

Chapter 2

Baseline and Lag-Ensemble Model

2.1 Deterministic baselines

Four target-week-excluding baseline rules are evaluated for each segment. If $y_{s,t}$ is the count for segment s in week t , the rules are:

$$\hat{y}_{s,t}^{(1)} = y_{s,t-1}, \quad (2.1)$$

$$\hat{y}_{s,t}^{(4)} = \frac{1}{4} \sum_{i=1}^4 y_{s,t-i}, \quad (2.2)$$

$$\hat{y}_{s,t}^{(8)} = \frac{1}{8} \sum_{i=1}^8 y_{s,t-i}, \quad (2.3)$$

$$\hat{y}_{s,t}^{(52)} = y_{s,t-52}. \quad (2.4)$$

The trailing eight-week arithmetic mean is selected by overall MAE. It covers 99.73% of the recorded backtest and provides a strong, interpretable reference for the lag ensemble [6, 7].

Table 2.1: Baseline performance on the December 30, 2024–December 22, 2025 backtest window. Lower errors are better.

Method	Coverage	MAE	RMSE	Weighted MAE
Previous week	99.97%	0.6010	1.7764	4.5649
Previous year, same week	97.64%	0.6796	2.0772	5.2965
Trailing four-week mean	99.87%	0.5058	1.4537	3.7967
Trailing eight-week mean	99.73%	0.4929	1.4128	3.7023

2.2 Feature engineering

The model manifest records a broad, auditable feature context:

- calendar variables: year, month, ISO week, quarter, and sequential week index;
- count lags at 1, 2, 4, 8, and 52 weeks;
- trailing four- and eight-week mean, standard deviation, minimum, and maximum;
- prior segment count, cumulative total, and mean; and

- borough, precinct, offense type, and law category identifiers.

Only four numeric inputs enter the deployed prediction formula: the one-week lag, the 52-week lag, and the trailing four- and eight-week means. The remaining engineered fields are retained as audit context rather than silently presented as formula inputs. Missing lag or rolling inputs fall back to available prior segment history and then to zero [9].

2.3 Model identity and prediction rule

The reviewed estimator is `duckdb_lag_ensemble_regressor`, model version 1 and artifact version 1. It is a deterministic formula implemented with the Python standard library and DuckDB; it does not persist a binary scikit-learn estimator. This choice makes the release portable and permits a direct audit of every prediction.

Let $m_{4,s,t}$ and $m_{8,s,t}$ denote prior-only four- and eight-week means, and let $\ell_{1,s,t}$ and $\ell_{52,s,t}$ denote one- and 52-week lags. The forecast is:

$$\hat{y}_{s,t} = \max \{0, \lambda [m_{8,s,t} + \alpha(m_{4,s,t} - m_{8,s,t}) + \beta(\ell_{1,s,t} - m_{4,s,t}) + \gamma(\ell_{52,s,t} - m_{8,s,t})]\}. \quad (2.5)$$

The selected parameters are $\alpha = 0.25$, $\beta = 0.10$, $\gamma = 0.05$, and $\lambda = 1.0$. The terms have intuitive roles: the eight-week mean is the anchor, the four-week term introduces recent momentum, the one-week term adds a short-run correction, and the 52-week term contributes a modest seasonal adjustment. The outer maximum prevents negative count estimates.

Why this model fits the project

The estimator is intentionally conservative. It improves on a strong moving average without obscuring the calculation behind a serialized black-box model. This matches the project’s emphasis on aggregate analysis, reproducibility, auditability, and explicit responsible-use limits.

2.4 Training and evaluation chronology

Random train/test splits are not used. Parameter selection is evaluated on a time-ordered validation window, followed by a later held-out backtest. The fixed forecast uses all available prior aggregate history through the partial week beginning December 29, 2025.

Table 2.2: Model lifecycle and evaluation windows.

Field	Value and interpretation
Aggregate history	December 26, 2005–December 29, 2025; the first label is a week boundary, not 2005 event coverage.
Parameter validation	January 1–December 23, 2024; selection objective is validation RMSE.
Held-out backtest	December 30, 2024–December 22, 2025; the final partial bucket is excluded.
Fixed forecast week	January 5, 2026.
Artifact generated	July 5, 2026 at 12:40:05 UTC.
Training completion	Not independently recorded; artifact generation time must not be relabeled as training completion.

The target week precedes artifact construction. The output is therefore a fixed retrospective demonstration rather than a rolling or real-time service.

Chapter 3

Evaluation, Interpretation, and Governance

3.1 Backtest results

The lag ensemble generates predictions for all 437,144 backtest segment-weeks. Table 3.1 summarizes the recorded overall results.

Table 3.1: Selected baseline and lag-ensemble backtest metrics.

Model	Coverage	MAE	RMSE	Weighted MAE
Trailing eight-week baseline	99.73%	0.4929	1.4128	3.7023
Lag ensemble	100.00%	0.4894	1.3943	3.6555
Difference: ML – baseline	+0.27 pp	−0.0035	−0.0185	−0.0468

The ML model improves all three reported error metrics, but the improvement is small. Moreover, the baseline and ML summaries contain different numbers of predictions (435,942 versus 437,144), so the differences are descriptive rather than a matched-row causal comparison. They should not be reported as guaranteed performance gains for a selected borough, precinct, offense, or law category.

The error distribution is also heterogeneous. High-volume and volatile series, including several petit larceny and harassment precinct/offense segments, have substantially larger errors than the overall average [8].

3.2 Leakage controls and reproducibility

The pipeline applies the following controls:

- every lag and rolling statistic excludes the target week;
- validation precedes backtesting in calendar time;
- random splits are disabled;
- the latest partial week is excluded from the backtest;
- zero filling begins only after a segment’s first observation; and
- generated artifacts use deterministic ordering and stable, repository-relative references.

The canonical rebuild order is the cleaning stage, dashboard summary, baseline, ML model, hotspots, anomalies, and the browser projections. A complete reproduction requires the raw CSV whose checksum matches the recorded source snapshot. Changing the source checksum or the explicit review date creates a different analytical release.

3.3 Limitations

Interpretation limits

Reported complaint counts reflect reporting behavior, delay, revision, under-reporting, classification, and policy change. They are neither complete measures of harm nor causal truth. Model output is a point estimate and does not include a prediction interval, calibration analysis, filter-specific error, or formal drift monitoring.

Important technical limitations include:

- the latest input bucket contains only three days;
- post-appearance zero filling may confuse a genuine zero with an upstream reporting gap;
- the formula omits holidays, reporting-delay corrections, exogenous events, spatial spillover, and explicit structural-break terms;
- the reported improvement is based on one historical split;
- the model has no general retraining cadence or model-age service threshold; and
- browser forecast geography withholds 2,614 model rows that cannot be mapped safely to the reviewed precinct contract.

The browser-safe forecast subset contains 5,852 rows across 78 precincts. This retains 69.12% of model rows but 99.62% of predicted volume. Expected Change has a baseline for 5,848 rows; four rows lack sufficient history, while 3,051 valid zero baselines make percentage change mathematically undefined.

3.4 Privacy and responsible use

Sensitive suspect and victim demographic fields are excluded from the cleaned modeling projection and all forecast features. The model does not generate an individual risk score, a future-incident location, or a patrol, enforcement, deployment, intervention, or resource-allocation recommendation.

Appropriate use is limited to transparent, retrospective exploration of aggregate reported-complaint patterns and model error. Hotspot and anomaly severity must not be interpreted as offense seriousness, neighborhood danger, or policing priority.

Part II

Dashboard Design and Implementation

Chapter 4

Dashboard Architecture

4.1 Technology stack and deployment model

The dashboard is a local single-page application implemented with React 19, TypeScript 6, and Vite 8. Recharts renders analytical charts and Leaflet renders spatial layers. The application has no API, scheduler, update channel, or long-running analytical service. Instead, it loads committed, browser-safe artifacts produced by the Python and DuckDB pipeline [12].

The recommended local launcher is:

```
./run.sh
```

The launcher validates Node.js and npm, installs locked dependencies when necessary, and starts Vite at `http://127.0.0.1:4173/`. The supported Node range is `^20.19.0 || ^22.13.0 || >=24.0.0`, with npm 10 or newer.

4.2 Browser-safe data contracts

The application loads five staged artifacts:

Table 4.1: Dashboard artifacts and their primary consumers.

Artifact	Role
<code>public/data/overview.json</code>	Metadata, governance fields, filters, summaries, and anomaly projection.
<code>public/data/overview-cube.bin.gz</code>	Compressed weekly cube used for observed totals and charts.
<code>public/data/map.json</code>	Fixed retrospective hotspot snapshot and list/detail data.
<code>public/data/forecast-map.json</code>	Precinct-level fixed forecast and Expected Change values.
<code>public/data/precinct-spatial-reference.json</code>	Reviewed 78-feature official precinct geometry.

Every loader treats incoming JSON as unknown data and validates it before rendering. Checks include schema and application identity, artifact version, timestamps, date alignment, finite numbers, unique stable keys, deterministic ordering, arithmetic consistency, availability state, and aggregate-only privacy flags. Contradictory or malformed data fails closed.

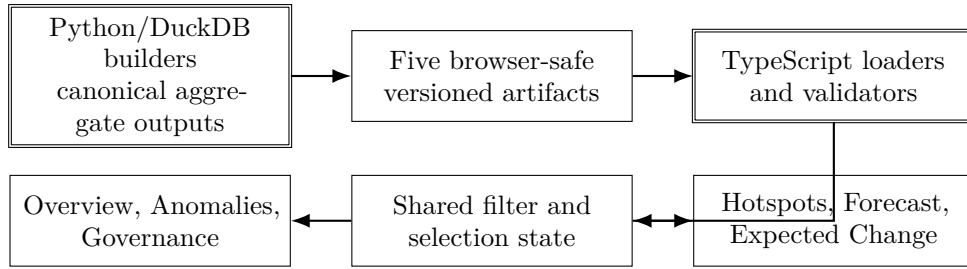


Figure 4.1: Dashboard data flow. The browser receives only validated aggregate artifacts; raw complaint rows never cross the browser boundary.

Optional sources remain independent. A missing forecast, hotspot, anomaly, or spatial artifact is not silently converted into an empty result or numeric zero. The UI distinguishes a valid zero, an available-but-empty result, a filter-produced empty state, missing history, an incompatible artifact, and a network failure.

4.3 Spatial reference and map resilience

Forecast and Expected Change use official `MultiPolygon` boundaries from the NYC Department of City Planning / NYC Open Data Police Precincts dataset, identifier `y76i-bdw7`, edition 26B. The spatial contract covers all 78 forecast precinct keys exactly once. It does not substitute complaint-derived hotspot centroids for administrative boundaries.

Raster tiles provide visual context but are not required for analytical access. If CARTO tiles fail, vector polygons or points, filters, legends, lists, and detail panels remain available. Geometry is withheld after a declared 120-day review window unless a new edition is reviewed; forecast list and detail values remain accessible even when polygons are stale.

Chapter 5

Analytical Views and Interaction Design

5.1 Overview

The Overview view combines a shared filter toolbar with metric cards, a weekly volume series, borough and offense comparisons, law-category totals, and a priority-signal table. The default complete-week window contains 567,306 reported rows, a recent four-week change of -12.8% , 26 high or critical hotspots, 645 high or critical anomalies, and a next-week point estimate of 8,168.3. These values are contextual summaries, not operational alerts.

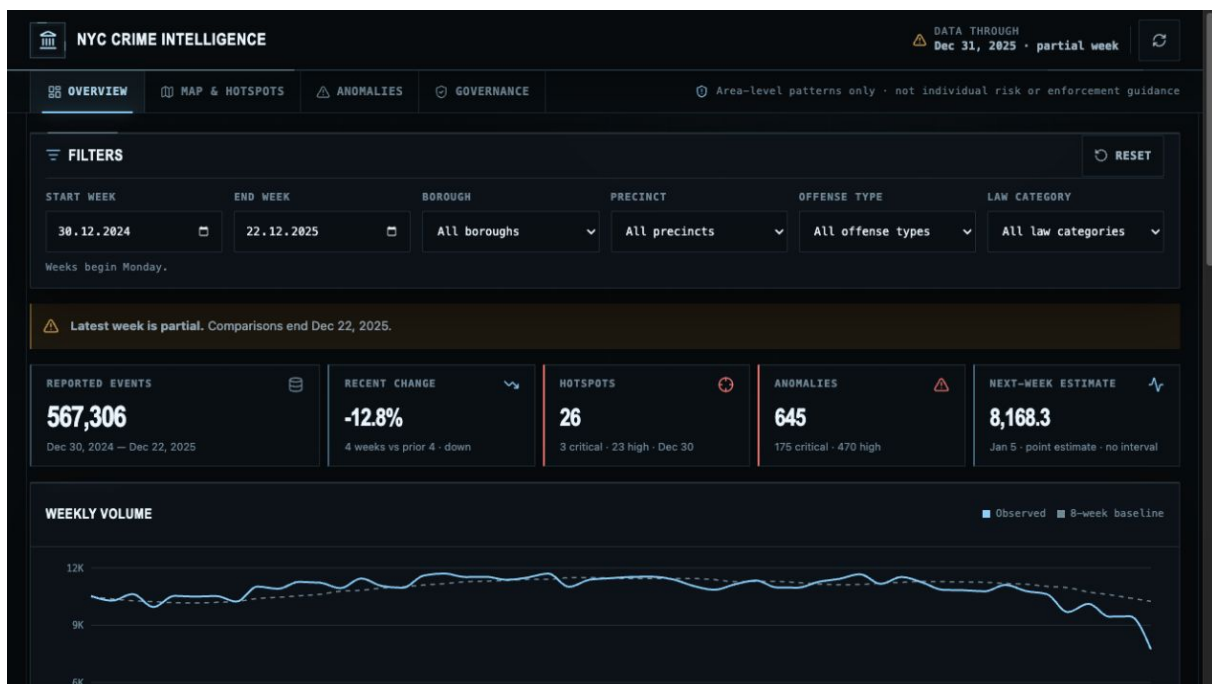


Figure 5.1: Overview interface with shared filters, aggregate metrics, partial-week warning, and observed-versus-baseline weekly trend.

Date filters select inclusive Monday-starting buckets. The recent-change card uses up to four selected complete weeks and compares them with the directly preceding equal-length window; partial weeks are excluded from this comparison.

5.2 Map workspace: Hotspots, Forecast, and Expected Change

The Map workspace contains three analytical modes under one set of categorical filters.

5.2.1 Hotspots

Hotspots are a fixed retrospective snapshot dated December 30, 2025. The method compares a recent 30-day count with a non-overlapping 365-day baseline; 7- and 90-day counts provide recency context. The composite score is:

$$0.35D + 0.25L + 0.20S + 0.15R + 0.05Q, \quad (5.1)$$

where D is density, L baseline lift, S share increase, R recency, and Q coordinate quality. The reviewed output contains 396 signals: 36 precinct and 360 grid records, of which 26 are high or critical [10].

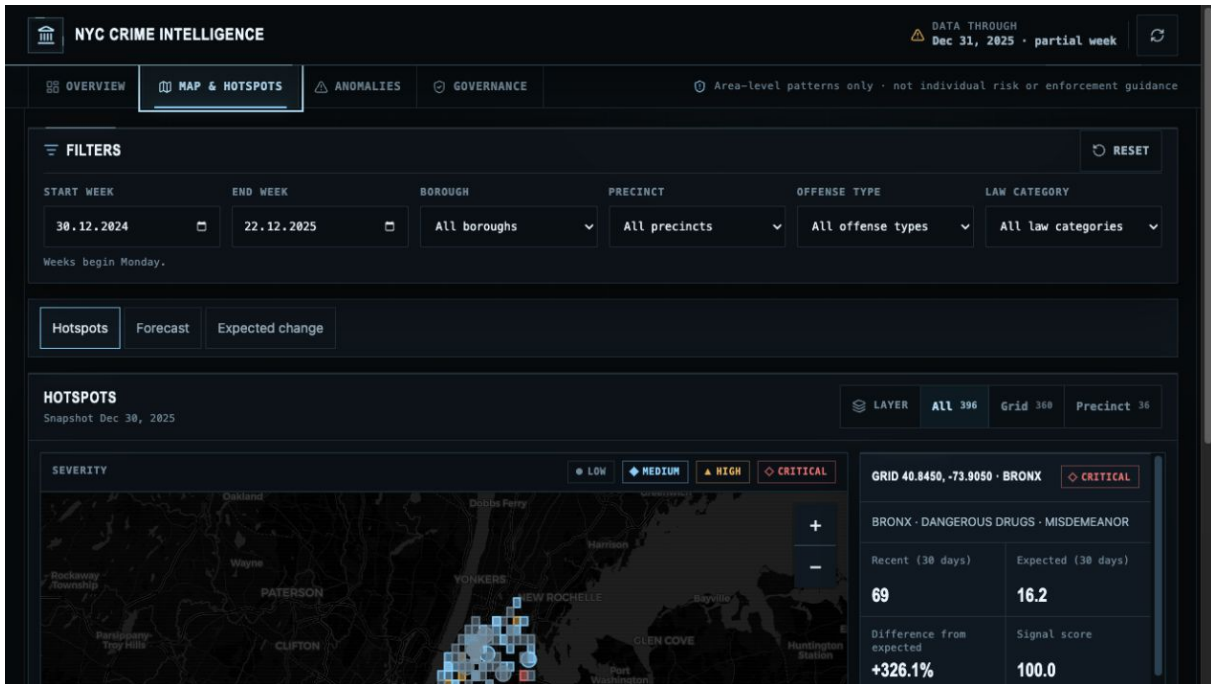


Figure 5.2: Hotspot mode with severity legend, aggregate map markers, and a synchronized detail panel. A full list/detail path is available below the map.

Grid points are 0.01-degree cell centers, and precinct points are aggregate centroids. Neither is an exact complaint location. The grid is not equal-area, and hotspot scores do not measure causality, offense seriousness, neighborhood danger, or action priority.

5.2.2 Forecast

Forecast mode aggregates filtered model rows to one point estimate per precinct for the fixed week beginning January 5, 2026. Polygons, list entries, and detail panels share one selection state. Date filters only determine whether this fixed horizon is compatible with the selected context; they do not create new forecast dates.

5.2.3 Expected Change

Expected Change displays the forecast minus the prior-only trailing eight-week arithmetic mean. Baselines are aggregated only when every contributing row has sufficient history. If any required

baseline is unavailable, change is withheld instead of treating missing history as zero. A valid zero baseline remains zero, and its percentage change is shown as undefined rather than infinite.

5.3 Anomalies

Anomalies are already-observed weekly aggregates that exceed a prior-only historical expectation. The detector uses trailing 8- and 13-week means, a 13-week standard deviation, and a 26-week median and median absolute deviation. When aligned, the expected value can use a leakage-safe model backtest estimate; otherwise it uses the prior 13-week mean.

The analytical output contains 89,362 flagged rows. The browser projection retains 10,378 high or critical records, with 645 in the default complete-week window. Rows expose actual, expected, residual, score, source, and severity. They are retrospective deviations, not probabilities or forecasts [11].

5.4 Governance

Governance is filter-independent and reconciles allowlisted metadata from the same dashboard artifacts. It presents:

- source coverage and row populations;
- overlapping quality flags and retained UNKNOWN values;
- model identity, versions, validation windows, artifact readiness, and lifecycle timestamps;
- known analytical and product limitations; and
- responsible-use and prohibited-use boundaries.

The view deliberately separates the source-data horizon, artifact generation time, and the unavailable independent training-completion time. It does not manufacture a generic health label or infer facts that the project did not record.

Chapter 6

Accessibility, Verification, and Operations

6.1 Accessible and responsive behavior

The dashboard uses native landmarks, headings, controls, lists, and status semantics. Map views have a complete list/detail path so an analyst does not need to operate the map with a pointer. Anomaly and spatial list rows are native buttons; visible selection, detail content, and polite status messages share a stable identity.

Severity is communicated through text and shape as well as color. Touch targets have a 44-pixel minimum, visible focus is preserved, and desktop, tablet, and mobile layouts avoid page-level horizontal overflow. Reduced-motion preferences remove or simplify transitions, while structural borders and solid-color fallbacks preserve content where blur or transparency are unavailable.

Automated tests cover native button activation, focus visibility, and selection synchronization. The remaining documented task is a final manual repetition of precinct-list keyboard activation on the intended browser and platform.

6.2 Automated verification

The repository-wide command `./scripts/verify_local.sh` checks Python contracts, documentation and repository hygiene, locked frontend installation, ESLint, Vitest, TypeScript compilation, the Vite production build, the production dependency audit, whitespace, and local port hygiene.

The reviewed acceptance pass on July 19, 2026 completed:

- 136 Python contract tests;
- 214 Vitest tests;
- ESLint and the TypeScript/Vite production build;
- production dependency audit;
- privacy, path, notebook, history, documentation, and whitespace checks; and
- desktop, tablet, mobile, state, tile-failure, console, and network review scenarios.

An optional full-data integration mode rebuilds the Overview, Map, Forecast Map, and compressed cube outputs from ignored local inputs and compares them exactly with the committed browser-safe artifacts.

6.3 Refresh and release workflow

The analytical dependency order is fixed. A full refresh runs cleaning, summary construction, baseline forecasting, ML forecasting, hotspots, anomalies, Overview projection, Map projection, Forecast Map projection, and spatial-reference projection. The explicit cleaning review date must advance only with a deliberate source refresh.

Canonical artifacts under `data/processed/` and their staged counterparts under `dashboard/public/data/` must remain byte-identical. Browser JSON must not be hand-edited, source-derived dates must not be replaced with wall clock time, and paths outside the declared repository root are rejected by the builders.

6.4 Dashboard limitations

The dashboard improves transparency but cannot eliminate the limitations of its source and model:

- the forecast is fixed and retrospective rather than live;
- all forecasts are point estimates without uncertainty intervals;
- hotspot thresholds are fixed and grid cells are unequal in area;
- precinct geometry depends on a separately reviewed quarterly source;
- raster context depends on an external tile provider; and
- overall historical errors do not guarantee accuracy for active filters.

The interface addresses these limits through explicit dates, partial-week warnings, model and baseline context, distinct unavailable states, a Governance view, aggregate-only data contracts, and repeated responsible-use language.

Conclusion

NYC Crime Intelligence demonstrates a coherent path from a large public source dataset to an auditable forecasting artifact and a defensive browser interface. Its strongest design decisions are the deterministic cleaning contract, prior-only feature construction, time-ordered evaluation, transparent lag formula, explicit baseline comparison, and strict separation between aggregate analytics and person-level or operational decision-making.

The lag ensemble produces a measurable but modest error reduction. That result supports the project's restrained product posture: the model is useful as an explainable retrospective demonstration, but it does not justify real-time or high-stakes use. The dashboard carries this conclusion into the interface by preserving time semantics, exposing limitations, distinguishing unavailable states from zero, and keeping all complaint-level and demographic information out of the browser.

Future technical work should prioritize matched-row model comparisons, prediction intervals and calibration, holiday and reporting-delay features, drift monitoring, a documented retraining policy, equal-area or better justified spatial methods, and completion of the final manual accessibility review.

Appendix A

Reproduction Commands

A.1 Python environment

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

A.2 Full analytical rebuild

```
.venv/bin/python src/data/build_clean_dataset.py \
  --as-of-date 2026-07-04
.venv/bin/python src/analytics/build_dashboard_summary.py
.venv/bin/python src/models/build_baseline_forecast.py
.venv/bin/python src/models/build_ml_forecast.py
.venv/bin/python src/analytics/build_hotspots.py
.venv/bin/python src/analytics/build_anomalies.py
.venv/bin/python src/analytics/build_dashboard_overview.py
.venv/bin/python src/analytics/build_dashboard_map.py
.venv/bin/python src/analytics/build_dashboard_forecast_map.py
.venv/bin/python \
  src/analytics/build_dashboard_precinct_spatial_reference.py
```

A.3 Verification and dashboard startup

```
./scripts/verify_local.sh
./scripts/verify_local.sh --full-data
./run.sh
```

Bibliography

- [1] NYC Crime Intelligence Project. *Repository README*. `README.md`, reviewed July 19, 2026.
- [2] NYC Crime Intelligence Project. *Final Project Report*. `reports/final_project_report.md`, July 19, 2026.
- [3] NYC Crime Intelligence Project. *NYPD Complaint Data Historic Provenance Note*. `data/source/nyc_open_data/nypd_complaint_data_historic.md`.
- [4] NYC Crime Intelligence Project. *Cleaning and Aggregation Report*. `reports/cleaning_report.md`, July 4, 2026.
- [5] NYC Crime Intelligence Project. *Data Quality Report*. `reports/data_quality_report.md`.
- [6] NYC Crime Intelligence Project. *Baseline Forecast Model Report*. `reports/baseline_model_report.md`, July 5, 2026.
- [7] NYC Crime Intelligence Project. *Baseline Forecast Model Manifest, Artifact Version 1*. `models/baseline_forecast/model_manifest.json`.
- [8] NYC Crime Intelligence Project. *ML Forecast Model Report*. `reports/ml_model_report.md`, July 5, 2026.
- [9] NYC Crime Intelligence Project. *Weekly Forecast Model Manifest, Model Version 1 and Artifact Version 1*. `models/weekly_forecast/model_manifest.json`.
- [10] NYC Crime Intelligence Project. *Hotspot Methodology*. `reports/hotspot_methodology.md`.
- [11] NYC Crime Intelligence Project. *Anomaly Methodology*. `reports/anomaly_methodology.md`.
- [12] NYC Crime Intelligence Project. *Dashboard Architecture and Operations Guide*. `dashboard/README.md`, reviewed July 19, 2026.
- [13] City of New York. *NYPD Complaint Data Historic (qgea-i56i)*. <https://data.cityofnewyork.us/Public-Safety/NYPD-Complaint-Data-Historic/qgea-i56i>.